

Volatility Model Recommendations for Individuals with Varying Financial Backgrounds

520380244, 520444052, 520487336, 510651642, 520020672

Table of contents

1	Executive Summary	2
2	Introduction	2
3	Methods	3
3.1	Data Collection & Processing	3
3.2	Model Development	4
3.3	Performance Metric: Accuracy	5
3.4	Performance Metric: Robustness	5
3.5	Performance Metric: Interpretability	6
3.6	Model Evaluation	7
4	Results & Evaluation	7
4.1	Accuracy	8
4.2	Robustness	8
4.3	Interpretability	9
4.4	Model Recommendations	9
5	Discussion & Conclusion	10
6	Appendix	11
7	Student Contributions	14
	References	15

1 Executive Summary

This study explores the critical differentiation in performance metric prioritised by various stakeholders in the domain of volatility prediction. It is evident that traders and interested individuals have distinct preferences for attributes in prediction models due to their unique requirements and purposes. Traders (technical stakeholder) generally place a higher emphasis on accuracy and robustness due to the necessity for rapid, reliable forecasts in a dynamic trading environment. Conversely, interested individuals (general stakeholder) may favour interpretability and accuracy to gain a clearer understanding of the variables that influence volatility forecasts. Our research aims to delineate these preferences and provide practical recommendations for each stakeholder group enhancing decision-making processes. This addresses the limitations of the traditional one-size-fits-all methodology in volatility prediction, presenting a more refined perspective that accommodates the diverse needs of users.

A variety of models were employed in this study, including Linear Regression (LR), Heterogeneous Autoregressive (HAV), Random Forest (RF), and Support Vector Machine (SVM). These models were assessed based on several performance metrics such as accuracy, robustness and interpretability to provide a comprehensive evaluation of model performance. In accuracy, RF performed best, followed by SVM, LR then HAV. In Robustness, SVM performed best, followed by RF, LR and HAV performing worst. In interpretability, LR was reported the most interpretable model, closely followed by RF. As such, we recommend the RF model for our technical stakeholder, and the LR model for general stakeholders.

The practical relevance of this analysis lies in its ability to equip traders and interested individuals with insights that align closely with their operational objectives and risk profiles. By tailoring volatility prediction models to the specific needs of different user groups, this research supports more informed decision-making, potentially leading to improved financial outcomes and greater confidence in the use of predictive analytics in investment strategies.

2 Introduction

This study aims to predict stock volatility using various predictive models, focusing on accuracy, robustness, and interpretability. The motivation for this project stems from the need to offer appropriate volatility prediction models to cater for the needs of diverse users. These are two primary stakeholder groups: traders and interested individuals with varying financial backgrounds.

Predicting stock volatility is a complex task that requires models capable of delivering accurate, robust, and interpretable predictions. For stakeholders with strong financial backgrounds, such as traders, the priority lies in selecting models that ensure high accuracy, robustness, and interpretability. These models are essential for effective risk management and strategic

decision-making. Conversely, interested individuals who may not have extensive financial expertise require models that are not only accurate but also interpretable and easy to use.

The study’s multidisciplinary approach incorporates elements of finance and data science to provide comprehensive insights into model selection and application. The financial context emphasises the necessity of accurate and robust models for effective risk management and informed investment strategies. Data science provides the tools and methodologies for processing large datasets, engineering features, and evaluating model performance.

By developing, evaluating and recommending models, this study aims to bridge the gap between these diverse requirements of each stakeholder group. Ultimately, this will aid stakeholders in making more informed decisions.

3 Methods

3.1 Data Collection & Processing

Training and testing data was obtained by taking a subset of the additional datasets: order book feature and order book target. Training and testing data was obtained by taking a subset of the additional datasets: order book feature and order book target. These datasets include order book snapshots of 10 stocks over multiple 1-hour intervals, occurring between January and October 2021. There are 11 columns in the datasets:

1. `stock_id`: unique ID of a stock
2. `time_id`: unique ID of a 1-hour time-bucket. The time IDs are sequential and consistent across all stocks.
3. `seconds_in_bucket`: number of seconds from the start of the time bucket
4. The remaining columns are the highest (lowest) two bid (ask) prices and their sizes.

The order book feature dataset contains data in the first 30 minutes of each time ID, while the order book target dataset contains data in the last 30 minutes.

The stock with ID 8382 was randomly selected from the 10 available stocks in the datasets. The time IDs were chosen by randomly selecting one day in each of the four quarters. The subsets of the two datasets were then combined on matching time IDs to obtain a complete dataset containing full 1-hour data of each selected time ID in sequential order.

Additional statistics were derived from existing order book data to reflect stock valuation and market liquidity, which is known to be correlated with volatility (Bennett, Justin 2023). These included weighted average price, bid-ask spread, bid spread, ask spread, number of orders,

volume imbalance, and log-returns. The data was then aggregated over non-overlapping 10-minute intervals to calculate the realised volatility and realised quarticity, as well as the mean and variance of the computed statistics, which would be the potential features for regression.

3.2 Model Development

The results of the backward selection based on AIC suggested that significant features for predicting realised volatility were mean and variance of bid-ask spread ($\overline{BAS}, \sigma_{BAS}^2$), variance of number of orders (σ_{Order}^2), variance of volume imbalance (σ_{VolIm}^2), and variance of bid spread ($\sigma_{BidSpread}^2$).

In the LR model, the realised volatility of period t was regressed on the observed covariates of the period $t - 1$:

$$\widehat{RV}_t = b_0 + b_1 \times \overline{BAS}_{t-1} + b_2 \times \sigma_{BAS_{t-1}}^2 + b_3 \times \sigma_{Order_{t-1}}^2 + b_4 \times \sigma_{VolIm_{t-1}}^2 + b_5 \times \sigma_{BidSpread_{t-1}}^2$$

The model was fitted using weighted least squares estimation such that the data points were exponentially weighted by their recency:

$$\min_{b_0, b_1, b_2, b_3, b_4, b_5} \sum_{t=2}^T w_t (RV_t - \widehat{RV}_t)^2$$

where $w_t = 0.8^{T-t}$

The HAV model directly used observed realised volatility of previous periods to predict the realised volatility of period t :

$$\widehat{RV}_t = b_0 + b_1 \times RV_{t-1} + \frac{b_2}{5} \sum_{i=1}^5 RV_{t-i}$$

Weighted least squares (WLS) was used to estimate the model coefficients:

$$\min_{b_0, b_1, b_2} \sum_{t=6}^n w_t (RV_t - \widehat{RV}_t)^2$$

where $w_t = RV_{t-1} / \sqrt{RQ_{t-1}}$, and $RQ_t = \frac{M}{3} \sum_{i=1}^M r_{t_i}^4$ is the realised quarticity.

Although the above two statistical regression models are simple and computationally efficient, they assume a linear relationship between realised volatility and the covariates. Hence, two machine learning models, RF and SVM, were also developed to capture any underlying non-linear relationships. RF and SVM were both trained on the same set of features as the LR model. When training the RF model, exponential weights, $w_t = 0.8^{T-t}$, were assigned to the data points based on their recency. However, unlike LR model, the weights were only used in sampling data to grow each tree and not in any other calculation. This enabled the

RF model to capture time information of the data by giving more recent observations a higher probability of being sampled.

The time series plots suggested that the realised volatility time series was non-stationary, and the underlying patterns appeared to change over time. In order to maintain prediction accuracy, it is crucial that the models continuously adapt to these dynamic changes. Thus, rolling window forecasting was implemented for model assessment. The process involved training the models on a 2-hour window of data and using them to predict the realised volatility of the next 10-minute period. After calculating the accuracy measures, the window was incremented by 10 minutes, and the process was repeated, continuing throughout a day. For each quarter, rolling window forecasting was performed on the randomly selected day.

A rolling window size of two hours was chosen based on the market making context. That is, the chosen stock belongs in an environment where price changes rapidly. Due to the mean-reverting nature of financial spreads, a two-hour window is long enough to capture the reversion dynamics (Stübinger and Endres 2018). As such, a two-hour window is effective in capturing and responding to short-term fluctuations without being oversensitive to random price movements or under-sensitive to reversion tendencies.

3.3 Performance Metric: Accuracy

The accuracy of the models was evaluated using four different measures of prediction errors. Mean Squared Error (MSE) and Root Mean Squared Percentage Error (RMSPE) penalise large errors, which is particularly important for avoiding significant financial risks in volatility prediction. However, due to their sensitivity to outliers, more robust measures, Mean Absolute Error (MAE) and Empirical Quasi-Likelihood (QLIKE), were also considered. Since the distributions of the error metrics were highly skewed with large outliers, the model accuracy was compared and ranked based on the medians. For each model, an accuracy score was computed by averaging the normalised medians of the four measures.

3.4 Performance Metric: Robustness

An additional performance metric of interest was the robustness of the model. A robust model is characterised by its ability to effectively manage diverse types of noise, variations, and uncertainties within the dataset, and to generalise across various scenarios and domains. In the context of volatility prediction, robustness was quantitatively assessed through the innovation of a metric termed “Adversarial Area”. This was derived from sensitivity analysis metrics such as adversarial accuracy and expected distance.

The Adversarial Area was computed as the area enclosed between the model’s initial prediction and three subsequent predictions. These latter predictions were tainted at contamination levels of 5%, 10%, and 15%, respectively.

Specifically, the original training data was randomly substituted with adversarial observations at the aforementioned levels. The substituted values remained logical and were confined to the range of values present in the original dataset.

The predictions were graphically represented, and the area between the initial model prediction and each subsequent level of contamination was calculated. These areas were recorded for each model and contamination level both as raw areas and as ranks. A robustness score was subsequently derived using a weighted-average method based on normalisation scores.

3.5 Performance Metric: Interpretability

Interpretability was defined as the ability to understand a model and how it arrives at its predictions. To innovate a metric for this, a [survey](#) was designed. The participants were surveyed on their knowledge of the four models that were created.

Firstly, participants were asked to class themselves based on their experience with finance and machine learning.

Then for each model, a brief description explaining how the algorithm behind the model worked was provided. To assess the interpretability of each model, participants were asked a series of questions to gauge their understanding:

1. Self-assessment: Participants rated their understanding of each model using a Likert scale from 0 (no understanding at all) to 1 (strong understanding).
2. Comprehension: Further multiple choice and short answer questions were provided to participants that tested their understanding of the inputs that go into each model, and if given sample inputs and model, if they could identify the correct output. Each incorrect answer was given a score of 0, whereas correct answers were given a score of 1.

The participants of the survey were then classed into two stakeholders: a “technical” stakeholder that had classed themselves as having “intermediate” or “advanced” experience with financial terminology/concepts, and a “general” that had classed themselves as having “no/minimal” experience with financial terminology/concepts. To calculate an interpretability score for each stakeholder, the following steps were taken:

1. For Technical Stakeholders, the scores from comprehension questions were given a weighting of 0.7, and self-assessment scores were given a weighting of 0.3. Therefore, the interpretability score for technical stakeholders was calculated as follows:

$$\text{Interpretability score} = 0.7 \times \text{Comprehension} + 0.3 \times \text{Self Assessment}$$

2. For General Stakeholders, the scores from comprehension and self-assessment questions were both given a weighting of 0.5. Therefore, the interpretability score for general stakeholders was calculated as follows:

$$\text{Interpretability score} = 0.5 \times \text{Comprehension} + 0.5 \times \text{Self Assessment}$$

The responses for each group were aggregated to find an average interpretability score for each model.

3.6 Model Evaluation

The models were evaluated using a weighted-average score of the three performance metrics:

$$\text{score} = w_1 \times s_1 + w_2 \times s_2 + w_3 \times s_3$$

where $\sum_{i=1}^3 w_i = 1$, and s_1, s_2, s_3 are the min-max normalised scores of accuracy, robustness and interpretability respectively.

The weights of the metrics were determined based on the stakeholder. For technical stakeholders, specifically traders, accuracy would be of the highest importance, followed by robustness. Hence, the weights of accuracy, robustness and interpretability were specified as 0.5, 0.3 and 0.2 respectively.

On the other hand, for general stakeholders who are simply interested in learning about volatility predictive models, a significantly large weight of 0.65 was placed on interpretability. Accuracy and robustness were weighted as 0.25 and 0.1 respectively.

4 Results & Evaluation

Table 1: Summary of performance metric scores

Model	Accuracy	Robustness	Technical Interpretability	General Interpretability
Linear Regression	3.9653	1.2500	0.8750	0.9018
HAV-WLS	3.3621	0.6713	0.6462	0.5982
Random Forest	5.2092	2.3660	0.8308	0.7946
Support Vector Machine	3.8697	3.4770	0.8000	0.7411

4.1 Accuracy

The evaluation of models was conducted using MSE, RMSPE, MAE, and QLIKE metrics to measure accuracy. When comparing the median errors illustrated in Table 2, RF dominated in all accuracy metrics, while HAV consistently performed the worst. According to Table 1, RF achieved the highest accuracy score of 5.21. LR and SVM were the second and third most accurate models with relatively similar accuracy scores of 3.97 and 3.86 respectively. HAV was the worst-performing model with a score of 3.36.

Table 2: Medians of accuracy measures

	MSE	QLIKE	MAE	RMSPE
Linear Regression	1.075e-07	5.178e-03	3.278e-04	1.057e-02
HAV-WLS	1.276e-07	6.312e-03	3.572e-04	1.287e-02
Random Forest	8.408e-08	3.510e-03	2.900e-04	7.481e-03
Support Vector Machine	1.250e-07	4.847e-03	3.536e-04	9.860e-03

4.2 Robustness

In the initial area comparison, which considered no contamination (0%) versus 5% contamination, the LR model exhibited superior performance, achieving an area of 0.000036. This was followed by the RF model with an area of 0.000054, the SVM model with an area of 0.000058, and the HAV model with an area of 0.000270.

During the second area comparison, assessing 0% versus 10% contamination, the SVM model demonstrated the best performance, recording an area of 0.001000. This was succeeded by the RF model with an area of 0.002249, the LR model with an area of 0.006164, and the HAV model with an area of 0.007830.

In the third area comparison, which evaluated 0% against 15% contamination, the SVM model again outperformed the others with an area of 0.004355, followed by the RF model with an area of 0.007777, the LR model with an area of 0.016953, and the HAV model with an area of 0.018466.

Figure 1 reflects the HAV robustness comparisons as a visual example of the adversarial areas.

The overall robustness rankings identified the SVM model as the most robust, followed by RF, LR, and HAV. Subsequent to these comparisons, the weighted average robustness scores for each model were calculated and are presented in Table 1.

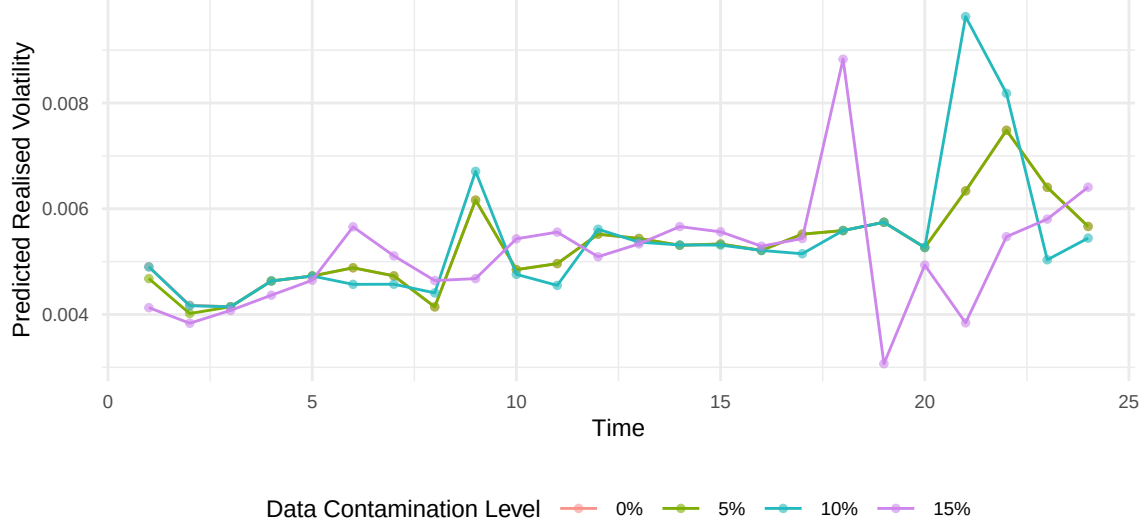


Figure 1: Adversarial area plot of HAV model

4.3 Interpretability

The interpretability of the four models were ranked the same for both general and technical stakeholders. According to Table 1, the LR model was the most interpretable, achieving the highest interpretability scores, and was followed closely by the RF model and then SVM. By far the least interpretable model for both stakeholders was the HAV model.

The survey results revealed that the majority of responders were confident that they had a good/strong understanding of the LR model. Responders also performed well in the comprehension questions on LR, with 25 out of 27 participants answered correctly.

On the other hand, more than half of the responses were recorded to have limited/basic understanding of the HAV model. This was also reflected in the comprehension questions results, as only 8 participants were able to answer all questions on HAV correctly.

4.4 Model Recommendations

The RF model was recommended for the technical stakeholder, as indicated by its highest weighted average score of 0.8425. It outperformed the other three models in terms of accuracy and was the second most robust model. As for the general stakeholder, the LR model achieved the highest weighted average score of 0.7523. The LR model was the most interpretable and achieved the second highest accuracy out of the four models.

It is important to note that the results would vary depending on the specified weights of the performance metrics. A Shiny App was created to assist the stakeholders in the model selection process by allowing them to specify the prioritised metrics and flexibly adjust the

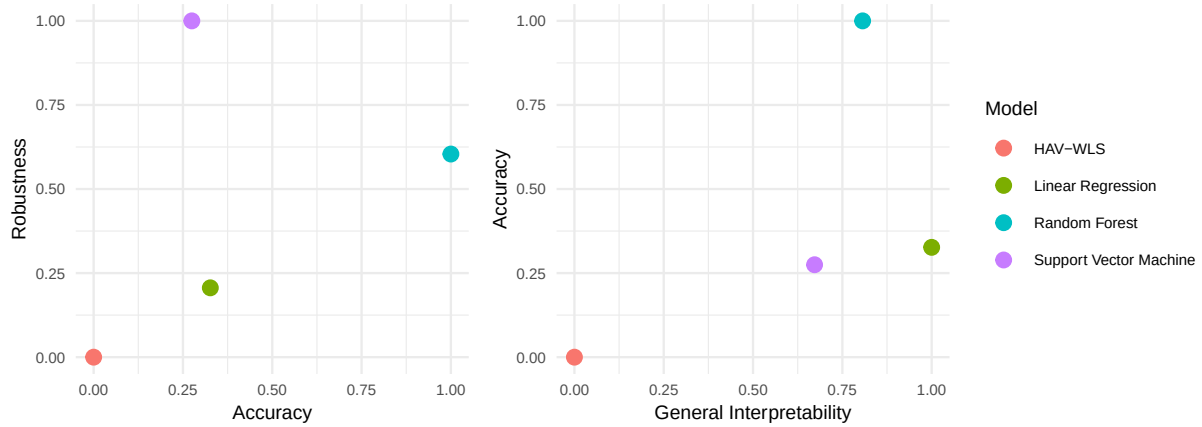


Figure 2: Scatter plots of performance metrics

corresponding weights. The weighted average scores of the models are calculated accordingly and displayed to the stakeholders. Additionally, the selected metrics are visualised on a scatter plot so that the performance of the models can be easily compared, as illustrated in Figure 2. Detailed results of each performance metric are also available in the [Shiny App](#).

5 Discussion & Conclusion

When creating the models, the underlying assumptions of each were not thoroughly considered. For example, a linear regression model assumes amongst other criteria, the normality of residuals.

Only four models were built using the data from one stock. The original intention was to build models using the ten stock data files in the additional datasets folder. These stocks all belonged to the technology sector or the investment banking industry. An issue with the approach taken is that the final models produced likely capture the idiosyncratic risk of the chosen firm and therefore, unique volatility patterns and characteristics of the particular firm, rather than the generalisable patterns across each sector.

Another explanatory variable considered was sentiment analysis. However, due to time constraints, this option was not pursued. Most intraday trading strategies found were based on exploiting short-term market inefficiencies and technical analysis (Thompson, Cedric 2024). There are some strategies listed by Thompson, Cedric (2024) that exploit the use of information (e.g., news, social media) however, there appears to be limited research conducted on the explanatory power of sentiment analysis and news on short-term volatility predictions.

This study aimed to provide technical and general stakeholders effective volatility models based on their diverse needs. Through measuring accuracy, robustness, and interpretability, we suggest the RF model for traders and LR for interested individuals. However, due to

the aforementioned limitations, we suggest two considerations in future research. Firstly, expanding the model development to the ten stocks in the additional datasets folder will enable a more comprehensive evaluation of the generalisability of the models across the technology and investment banking sectors. The model accuracy could also be further improved by performing hyperparameter tuning and more extensive feature selection.

Secondly, incorporating sentiment analysis into the models should be prioritised. Despite there being limited research has been conducted on the impact of sentiment analysis on financial volatility in the context of intraday trading horizons. However, an abundance of research suggests that sentiment analysis is a potent tool in predicting stock price movements for longer trading horizons. There may also be significant potential for sentiment analysis to enhance the accuracy of short-term volatility predictions. By conducting sentiment analysis, a deeper understanding of the impact of market sentiment on price movements and financial volatility can be unearthed.

6 Appendix

Appendix A: Code for computing accuracy score

```
# Normalise the median errors
# by dividing by sum of medians across the 4 models for given error metric
error_normalised = as.data.frame(lapply(median_acc,
                                         function(x) x/rowSums(median_acc)))
# Average the normalised medians of the metrics for each model
acc_score = as.data.frame(t(1/colMeans(error_normalised)))
colnames(acc_score) = c('Linear Regression', 'HAV-WLS',
                        'Random Forest', 'Support Vector Machine')

acc_score
```

Appendix B: Code for computing adversarial area

```
# Function to replace a percentage of volatility values with random values
replace_data <- function(data, percentage, lower = 0.004, upper = 0.008) {
  set.seed(2024)
  n_replace <- floor(nrow(data) * percentage) # Number of elements to replace
  idx <- sample(nrow(data), size = n_replace, replace = FALSE)

  # Replace the data at those indices with random values between lower and upper
  df = data
  for (i in 1:length(idx)) {
    df[idx[i], 2] <- runif(1, min = lower, max = upper)
  }
}
```

```

    }
    return(df)
}

# Function to fit models on contaminated data to make predictions
volatility_prediction_with_random_replacement <- function(percentage, model) {
  replaced_pred = list()
  for (i in 1:length(quarters)){
    replaced_vol = replace_data(quarters.vol[[i]], percentage)
    if(model == 'lm'){
      pred = fit_lm(replaced_vol, quarters.features[[i]])[[3]]
    } else if(model == 'hav'){
      pred = fit_HAV(replaced_vol, quarters.quarticity[[i]])[[3]]
    } else if(model == 'rf'){
      pred = fit_rf(replaced_vol, quarters.features[[i]])[[2]]
    } else if(model == 'svm'){
      pred = fit_SVM(replaced_vol, quarters.features[[i]])[[2]]
    }
    replaced_pred[[i]] = pred
  }
  return(replaced_pred)
}

# Function to calculate the area between two lines using the trapezoidal rule
calculate_area <- function(line1, line2) {
  abs_diff <- abs(line1 - line2)
  area <- trapz(abs_diff)
  return(area)
}

models = c('lm', 'hav', 'rf', 'svm')
models.pred = list(quarters.pred.lm, quarters.pred.HAV,
                   quarters.pred.rf, quarters.pred.svm)
robust_areas = list()

for(i in 1:length(models)){
  train_5_pred = volatility_prediction_with_random_replacement(0.05, models[i])
  train_10_pred = volatility_prediction_with_random_replacement(0.10, models[i])
  train_15_pred = volatility_prediction_with_random_replacement(0.15, models[i])
  quarters.pred = models.pred[[i]]
  # Averaging the 4 different predictions of the quarters

```

```

train_5_pred <- mapply(mean, train_5_pred[[1]], train_5_pred[[2]],
                      train_5_pred[[3]], train_5_pred[[4]], na.rm = TRUE)
train_10_pred <- mapply(mean, train_10_pred[[1]], train_10_pred[[2]],
                       train_10_pred[[3]], train_10_pred[[4]], na.rm = TRUE)
train_15_pred <- mapply(mean, train_15_pred[[1]], train_15_pred[[2]],
                       train_15_pred[[3]], train_15_pred[[4]], na.rm = TRUE)
train_0_pred <- mapply(mean, quarters.pred[[1]], quarters.pred[[2]],
                      quarters.pred[[3]], quarters.pred[[4]], na.rm = TRUE)
# Create dataframe for prediction values
df = lapply(list(pred = train_0_pred, train_5 = train_5_pred,
                train_10 = train_10_pred, train_15 = train_15_pred),
            function(x) data.frame(Time = 1:length(x), Volatility = unlist(x)))
combined.df = do.call(rbind, lapply(names(df),
                                     function(x) {cbind(df[[x]], Condition = x)}))
combined.df$Condition = factor(combined.df$Condition,
                              levels=c('pred', 'train_5', 'train_10', 'train_15'))
# Extracting the relevant data from the combined data frame
true_pred <- combined.df$Volatility[combined.df$Condition == 'pred']
train_5 <- combined.df$Volatility[combined.df$Condition == 'train_5']
train_10 <- combined.df$Volatility[combined.df$Condition == 'train_10']
train_15 <- combined.df$Volatility[combined.df$Condition == 'train_15']
# Calculating the areas
area.train_5 <- calculate_area(true_pred, train_5)
area.train_10 <- calculate_area(true_pred, train_10)
area.train_15 <- calculate_area(true_pred, train_15)
# Tabulating the results
robust.areas[[i]] = data.frame(
  `Comparison` = c("0% vs 5%", "0% vs 10%", "0% vs 15%"),
  `Area` = c(area.train_5, area.train_10, area.train_15)
)

names(robust.areas[[1]]) <- c("Comparison", "Area LR")
names(robust.areas[[2]]) <- c("Comparison", "Area HAV")
names(robust.areas[[3]]) <- c("Comparison", "Area RF")
names(robust.areas[[4]]) <- c("Comparison", "Area SVM")

# Dataframe of adversarial areas of all models
combined_area <- Reduce(function(x, y) merge(x, y, by = "Comparison", all = TRUE),
                        robust.areas)
comparison_order <- factor(combined_area$Comparison,
                          levels = c("0% vs 5%", "0% vs 10%", "0% vs 15%"))

```

```
combined_area <- combined_area[order(comparison_order), ]
combined_area
```

Appendix C: Code for computing robustness score

```
# Dataframe with adversarial areas of each model per column
data_to_normalise = combined_area[, 2:5]
# Normalise the areas
# by dividing it by the total area across the 4 models for given contamination level
area_normalised = as.data.frame(lapply(data_to_normalise,
                                       function(x) x/rowSums(data_to_normalise)))
# Sum up normalised areas for each model
sum_area_normalised = colSums(area_normalised)
# Invert results as smaller areas correspond to greater robustness
robust_score = as.data.frame(t(1/sum_area_normalised))
colnames(robust_score) = c('Linear Regression', 'HAV-WLS',
                          'Random Forest', 'Support Vector Machine')

robust_score
```

Appendix D: Code for computing final weighted average score

```
# Min-max normalisation of each metric score
norm_acc_score = (acc_score - min(acc_score))/(max(acc_score)-min(acc_score))
norm_robust_score = (robust_score - min(robust_score))/
                    (max(robust_score)-min(robust_score))
norm_tech_inter_score = (tech_inter_score - min(tech_inter_score))/
                        (max(tech_inter_score)- min(tech_inter_score))
norm_general_inter_score = (general_inter_score - min(general_inter_score))/
                           (max(general_inter_score)- min(general_inter_score))
# Technical stakeholder - accuracy > robustness > interpretability
tech_score = 0.5*norm_acc_score + 0.3*norm_robust_score
            + 0.2*norm_tech_inter_score
# General stakeholder - interpretability > accuracy > robustness
general_score = 0.25*norm_acc_score + 0.1*norm_robust_score
              + 0.65*norm_general_inter_score
```

7 Student Contributions

The following paragraph lists contributions of members. 520020672 created the RF model and related accuracy, drafted the initial Shiny App, and contributed to the model development

method in the report. 510651642 created a draft GARCH model, SVM model and composed model development method, accuracy results and introduction of the report. 520380244 conducted data processing, feature engineering, built LR and HAV models and related accuracy measures. 520380244 also created the Shiny app, and wrote the following sections in the report: data processing, model development, accuracy method, model evaluation, model recommendation results. 520444052 created and implemented robustness, in the report they wrote the methods and results for robustness and executive summary. 520444052 and 520380244 edited and formatted the report. 520487336 created the interpretability survey and implemented the scoring, in the report they wrote the methods, results for interpretability alongside the discussion and conclusion. 520487336 was also involved in providing relevant financial disciplinary research.

References

- Bennett, Justin. 2023. “How Are Market Liquidity and Volatility Related.” 2023. <https://dailypriceaction.com/blog/forex-liquidity-volatility/#:~:text=A%20volatile%20market%20is%20one,market%20can%20also%20become%20volatile>.
- Stübinger, Johannes, and Sylvia Endres. 2018. “Pairs Trading with a Mean-Reverting Jump–Diffusion Model on High-Frequency Data.” *Quantitative Finance* 18 (10): 1735–51. <https://doi.org/10.1080/14697688.2017.1417624>.
- Thompson, Cedric. 2024. “Rules for Picking Stocks When Intraday Trading.” 2024. <https://www.investopedia.com/day-trading/pick-stocks-intraday-trading/>.